Princípios de Lógica: Estruturas de Controle

Produzido por Prof. Veríssimo
Março / 2026

Fatec - Centro Paula Souza

1 Estrutura de Controle

Em lógica de programação atemos que tratar as entradas, processamentos e saídas com a utilização de variáveis constantes e operadores aritméticos para que consigamos solucionar problemas e transformá-los em programas. Imagine agora, a seguinte situação: um programa que apresente a média escolar de um aluno. Até aqui, muito simples, mas além de calcular a média, o programa deve apresentar se ele está aprovado ou reprovado segundo a análise de sua média. Observe que aqui será necessário verificar a média do aluno para então tomar uma decisão no sentido de apresentar a sua real situação: aprovado ou reprovado. Neste sentido, é importante que tenhamos o domínio de tratar "desvios comdicionais:

- Desvios condicionais simples
- Desvios condicionais encadeados

1.1 Desvio Condicional Simples

Para solucionar o problema proposto, é necessário trabalhar uma nova instrução: `se ...então...fim_se`. A instrução `se...então...fim_se` tem por finalidade tomar uma decisão. Sendo a condição **Verdadeira**, serão executadas todas as instruções que estejam entre a instrução `se...então` e a instrução `fim_se`. Sendo a condição ***Falsa***, serão executadas as instruções que estejam após o comando `fim_se`.

Diagrama de Blocos

Observe no diagrama a existência das letras **S** e **N**, além das linhas com seta indicando a direção do processamento, colocadas juntamente com o símbolo de Decisão. O **S** representa ***sim*** e está posicionado para indicar que um determinado bloco de operações será executado quando a condição atribuída for verdadeira. O **N** está para ***não*** e será executado quando a condição for *falsa*. O símbolo do losango, ou melhor dizendo, **Decisão** deverá ser utilizado em situações em que haja a necessidade de usar uma decisão dentro do programa.Uma decisão será tomada sempre com base em uma pergunta, como **RESPOSTA = "sim"**, e é esta pergunta que deverá estar indicada dentro do símbolo de losango, conforme demonstrado à figura 1.

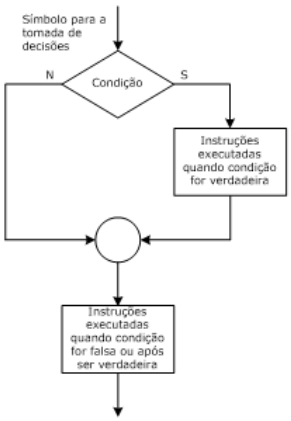


Figura 1: Exemplo de Decisão

Português Estruturado

```
se (condicao) entao
    <instru  es para condicao verdadeira>
fim_se
<instru  es para condicao falsa ou apos ser verdadeira>
```

Como um exemplo, considere o seguinte problema: "Ler dois valores numéricos, efetuar a adição e apresentar o seu resultado caso o valor somado seja maior que 10". Veja o diagrama de blocos e a codificação em português estruturado.

Algoritmo

Veja que fazemos os seguintes passos:

1. Conhecer dois valores incógnitos (estabelecer variáveis A e B);
2. Efetuar a soma dos valores incógnitos A e B, implicando o valor da soma na variável X;
3. Apresentar o valor da soma contido na variável X, caso o valor de X seja maior que 10.

Este algorimo é implementado no diagrama de blocos da figura 2.

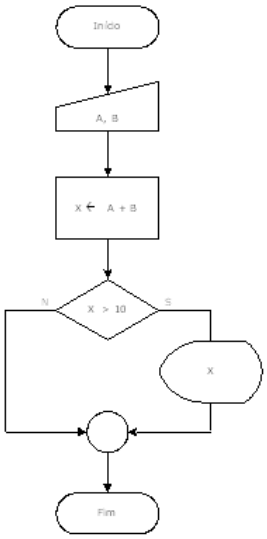


Figura 2: Estrutura `se...então...fim_se`

Desvio Condicional Composto

Há pouco você conheceu como fazer uso da instrução se `então...fim_se` do tipo simples. Agora você aprenderá a fazer uso da instrução `se então...senão...fim_se`, que sendo a condição **Verdadeira**, serão executadas todas as instruções que estejam posicionadas entre o `se...então` e a instrução `senão`. Sendo a condição **Falsa**, serão executadas as instruções que estejam entre o `senão` e a instrução `fim_se`.

Este estrutura é represenetada é implementado no diagrama de blocos da figura 3.

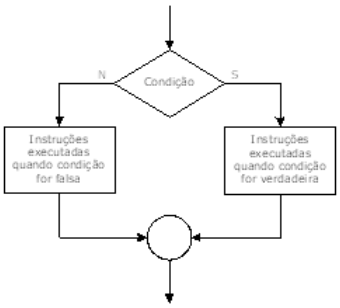


Figura 3: Representação da Estrutura `se...então...senao...fim_se`

Português Estruturado

```
se (condicao) entao
    <instrucoes para condicao verdadeira>
senao
    <instrucoes para condicao falsa>
fim_se
```

Algoritmo

Veja que fazemos os seguintes passos:

1. Conhecer dois valores (variáveis A e B);
2. Efetuar a soma dos valores A e B e implicar o valor da soma em X;
3. Verificar se X é maior ou igual 1O; caso sim, calcule R = X+5, senão calcule R = X-7.

Este algorimo é implementado no diagrama de blocos da figura 4.

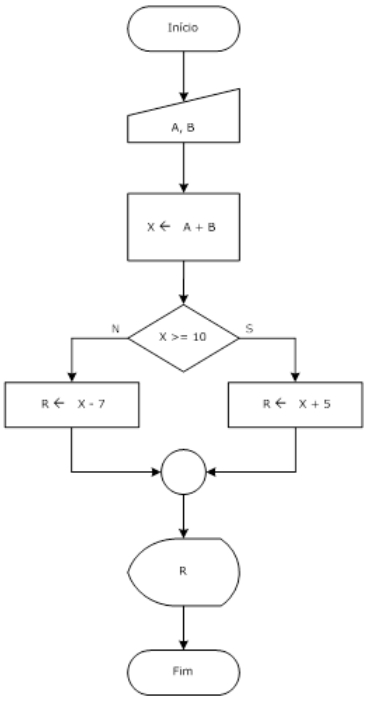


Figura 4: Exemplo de utilização da estrutura `se...então...senao...fim_se`

Observe que após a definição dos tipos de variáveis, é solicitada a leitura dos valores para as variáveis A e B, depois esses valores são implicados na variável **X**, a qual possui o resultado da adição dos dois valores. Neste ponto, é questionado no programa uma condição que permitirá imprimir o resultado da soma adicionado de 5, caso esta seja maior ou igual a 10, e não sendo, o programa apresentará o resultado subtraindo 7.

Desvios Condicionais Encadeados

Existem casos em que é necessário estabelecer verificação de condições sucessivas, em que uma determinada ação poderá ser executada se um conjunto anterior de instruções ou condições for satisfeito. Sendo a ação executada, ela poderá ainda estabelecer novas condições. Isto significa utilizar uma condição dentro de outra condição. Este tipo de estrutura poderá possuir diversos níveis de condição, sendo chamadas de aninhamentos ou encadeamentos.

Este estrutura é represenetada é implementado no diagrama de blocos da figura 5.

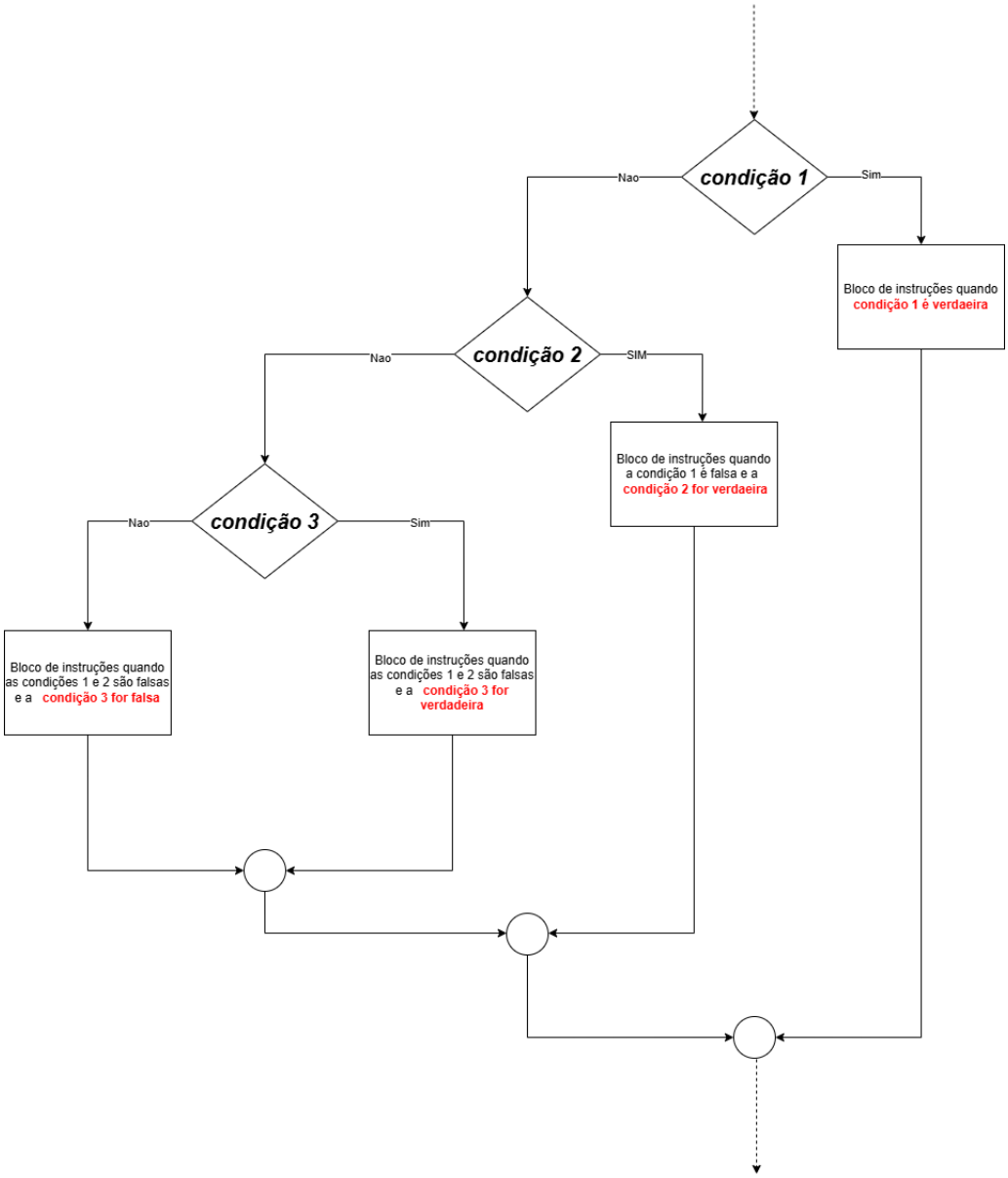


Figura 5: Representação da Estrutura de desvios condicionais encadeados

Português Estruturado

```
se (<condicao1>) entao
    <instrucoes para condicao 1 verdadeira>
senao
    se (<condicao2>) entao
         <instrucoes para condcao1 falsa e condicao2 verdadeira>
    senao
         se (<condicao3>) entao
            <instru  es para condicao1 e condicao2 falsas e
               condicao3 verdadeira>
         senao
            <instrucoes para condicao1 e condicao2 falsas e
               condicao3 falsa>
         fim_se
     fim_se
fim_se
```

Para um exemplo da utilização desta estrutura considere o seguinte problema (**utilizaremos somente 2 níveis de encadeamento**):

Descrição do problema

Elaborar um programa que efetue o cálculo do reajuste de salário de um funcionário. Considere que o funcionário deverá receber um reajuste de 15% caso seu salário seja menor que 500. Se o salário for maior ou igual a 500, mas menor ou igual a 1000, seu reajuste será de 10%; caso seja ainda maior que 1000, o reajuste deverá ser de 5%". Veja o algoritmo, diagrama de blocos e a codificação em português estruturado.

.....

Algoritmo

Perceba que o problema em questão estabelece três condições para calcular o reajuste do salário do funcionário, sendo :

- Salário $<$ 500, reajuste será de 15%
- Salário $>=$ 500, mas $<=$ 1000, reajuste será de 10%
- Salário $>$ 1000, reajuste será de 5%

Diagrama de Blocos

Este algorimo é implementado no diagrama de blocos da figura 6.

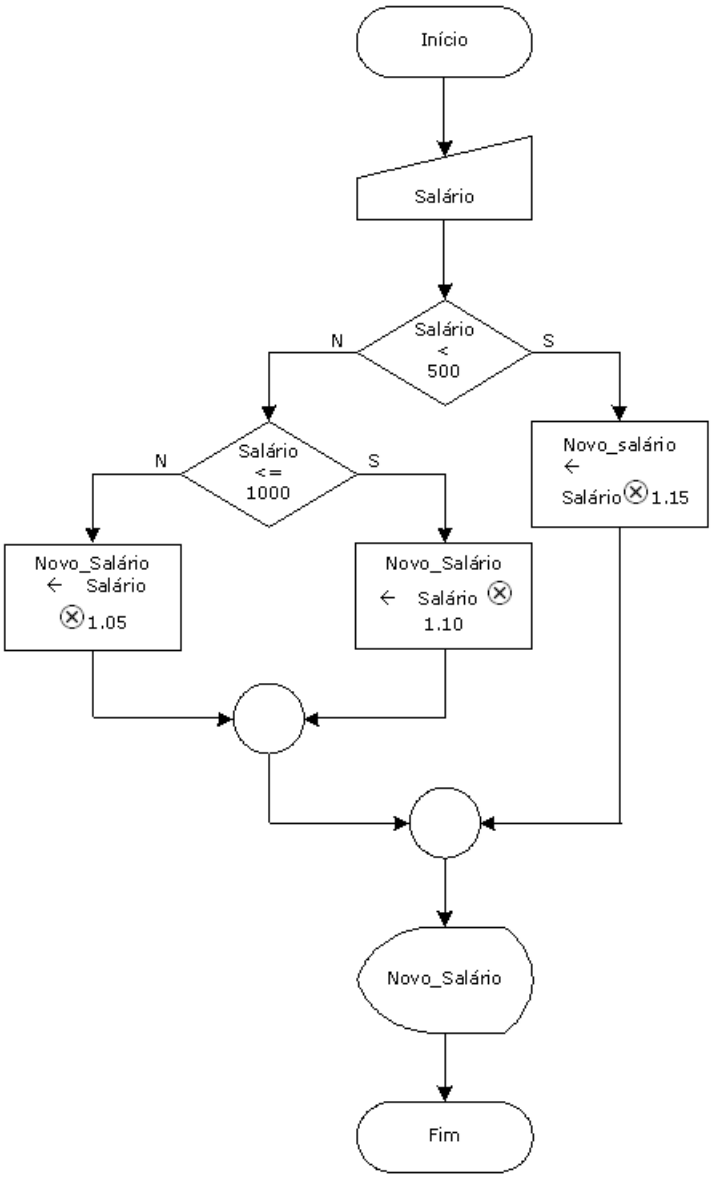


Figura 6: Exemplo de utilização de unma estrutura condicional encadeada

Observe que a referência da linha 5 do algoritmo abaixo não é escrita no diagrama de blocos e em português estruturado, uma vez que ela fica subentendida, ou seja, qualquer valor que não seja menor que 500 ou que não esteja situado na faixa de 500 a 1000 está, consequentemente, acima de 1000.

1. Definir uma variável para o salário reajustado: NOVO_SALÁRIO;
2. Ler um valor para a variável SALÁRIO;
3. Verificar se o valor de SALÁRIO $<$ 500, se sim reajustar em 15

4. Verificar se o valor de SALÁRIO <=1000, se sim reajustar em 10
5. Verificar se o valor de SALÁRIO> 1000, se sim reajustar em 5
6. Apresentar o valor reajustado, implicado em NOVO_ALÁRIO.

2 Estruturas de Repetição

Em diversas situações, precisamos que um bloco de instruções seja executado repetidamente até que uma determinada condição seja atingida ou enquanto ela for verdadeira. Essas estruturas são conhecidas como "laços"ou *loops*. Estudaremos as três principais formas: `para`, `enquanto...faça` e `faça...enquanto`.

2.1 Estrutura de Repetição: `para` (for)

A estrutura `para` é utilizada quando sabemos previamente o número de vezes que um conjunto de instruções deve ser executado. Ela utiliza uma variável de controle (contador) que é incrementada automaticamente a cada iteração.

Português Estruturado

```
para <variavel> de <valor_inicial> ate <valor_final> [passo <
   incremento>] faca
    <instrucoes a serem repetidas>
fim_para
```

Diagrama de Blocos

No diagrama de blocos, essa estrutura costuma ser representada por um hexágono alongado ou um símbolo de processamento que indica a inicialização, a condição de parada e o incremento da variável de controle.

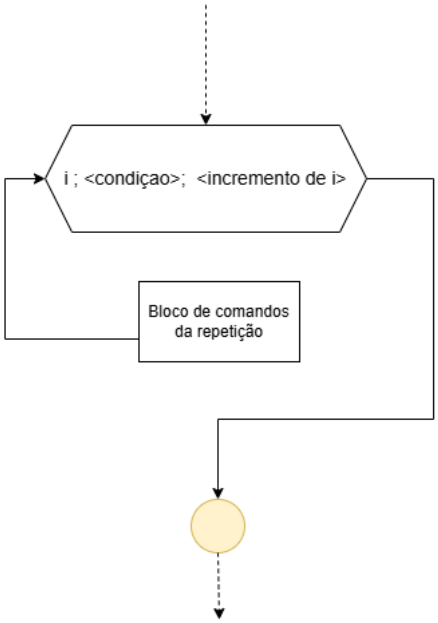


Figura 7: Representação da Estrutura `para`

2.2 Estrutura de Repetição: `enquanto...faça` (while)

A estrutura `enquanto` (conhecida como *pré-testada*) verifica a condição **antes** de executar o bloco de instruções. Se a condição for falsa logo de início, o bloco nunca é executado.

Português Estruturado

```
enquanto (condicao) faca
    <instrucoes a serem repetidas>
    <atualizacao da condicao>
fimespero
```

Importante

É fundamental que, dentro do laço, exista uma instrução que altere o valor da condição, caso contrário, o programa entrará em um **loop infinito**.

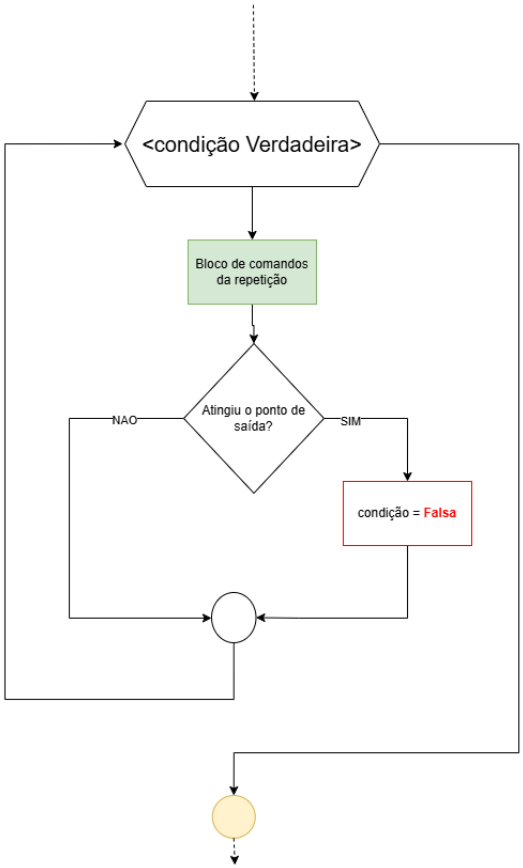


Figura 8: Representação da Estrutura `enquanto...faca`

2.3 Estrutura de Repetição: `faça...enquanto` (do...while)

Diferente da anterior, esta estrutura é *pós-testada*. Isso significa que o bloco de instruções é executado pelo menos uma vez, pois a verificação da condição ocorre apenas no **final** do laço.

Português Estruturado

```
faca
    <instrucoes a serem repetidas>
enquanto (condicao)
```

Comparativo

Enquanto no `enquanto...faça` o teste é feito no topo, no `faça...enquanto` o fluxo de execução garante a primeira passagem pelas instruções antes de decidir se repete o processo.

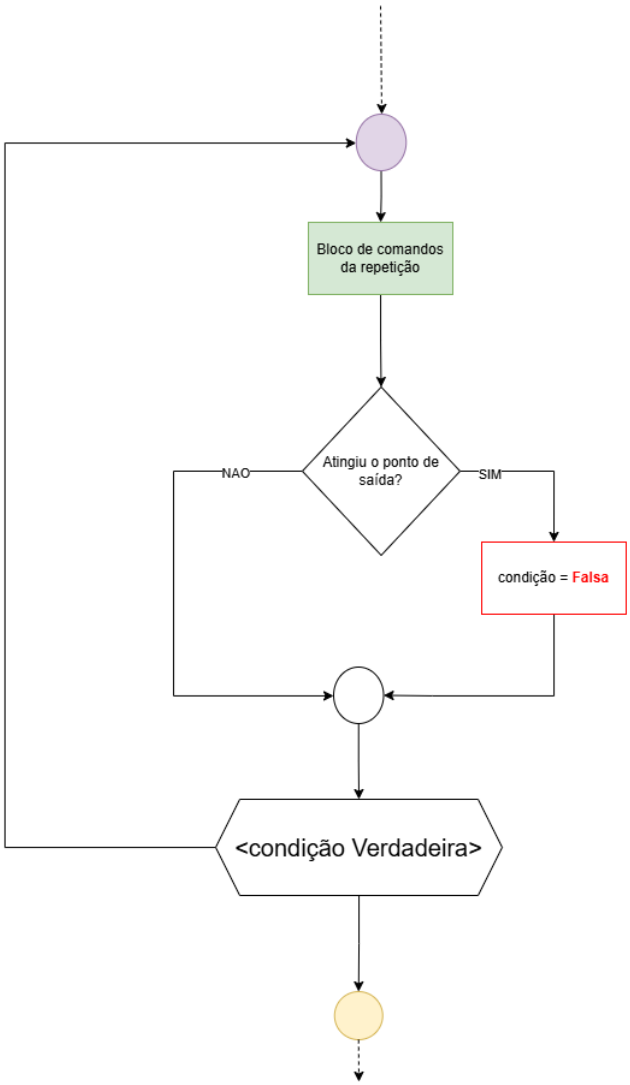


Figura 9: Representação da Estrutura `faca...enquanto`

Importante

Assim como no laço de repetição `enquanto...faça` É fundamental que, dentro do laço, exista uma instrução que altere o valor da condição, caso contrário, o programa entrará em um **loop infinito**.

3 Exemplos Práticos: Tabuada do 7

Para consolidar o entendimento das estruturas de repetição, apresentaremos o desenvolvimento de um programa que calcula e exibe a tabuada do número 7, utilizando as três variações de laços.

3.1 Exemplo com a Estrutura `para` (for)

A estrutura `para` é ideal para este caso, pois sabemos exatamente que a tabuada irá do multiplicador 1 até o 10.

Lógica do Algoritmo

Definimos um contador i iniciando em 1, com interrupção em 10 e incremento unitário. A cada volta, calculamos $R = 7 \times i$.

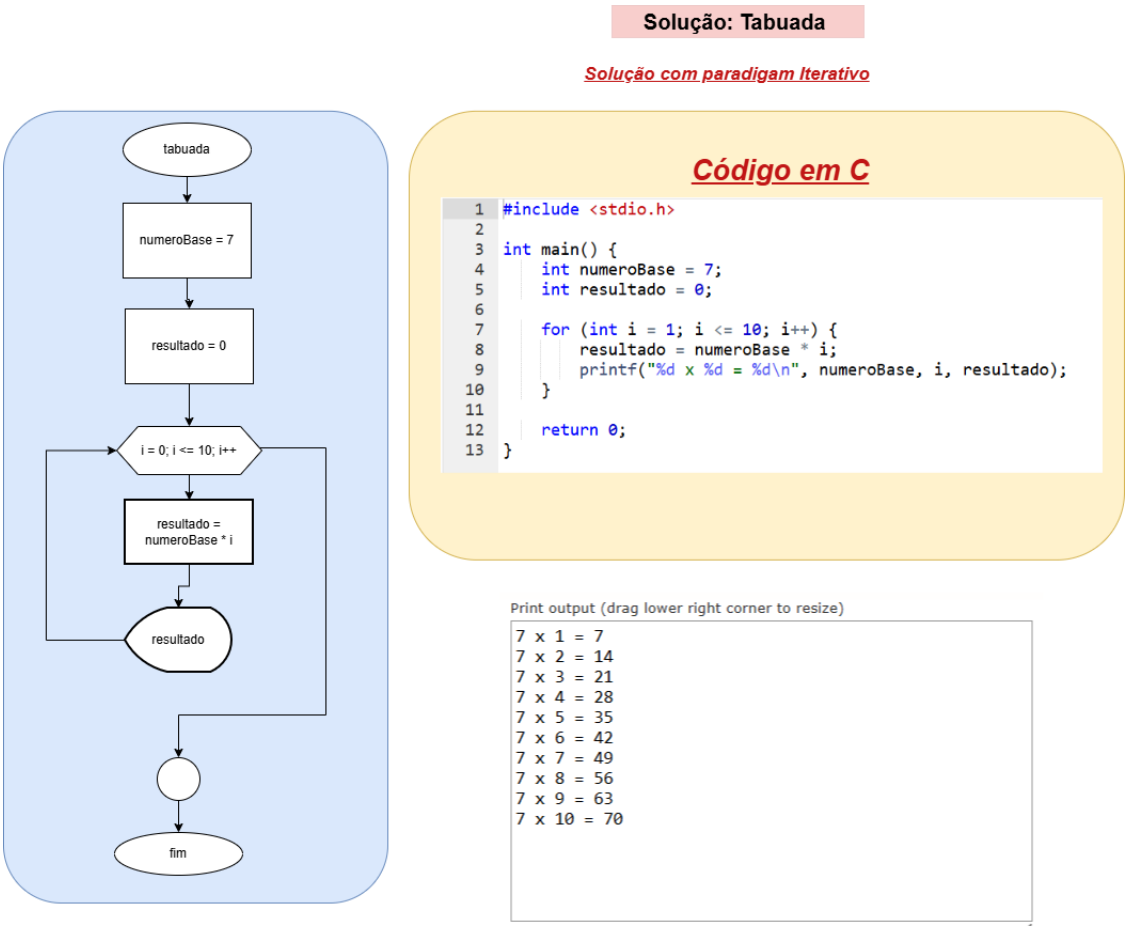


Figura 10: Diagrama, Código C e Resultado - Laço `FOR`

3.2 Exemplo com a Estrutura `enquanto` (while)

Neste modelo, a variável de controle deve ser inicializada antes do laço e incrementada manualmente dentro do bloco de instruções.

Português Estruturado

```
i <- 1
enquanto (i <= 10) faca
    resultado <- 7 * i
    escrever "7 x ", i, " = ", resultado
    i <- i + 1
fimespero
```

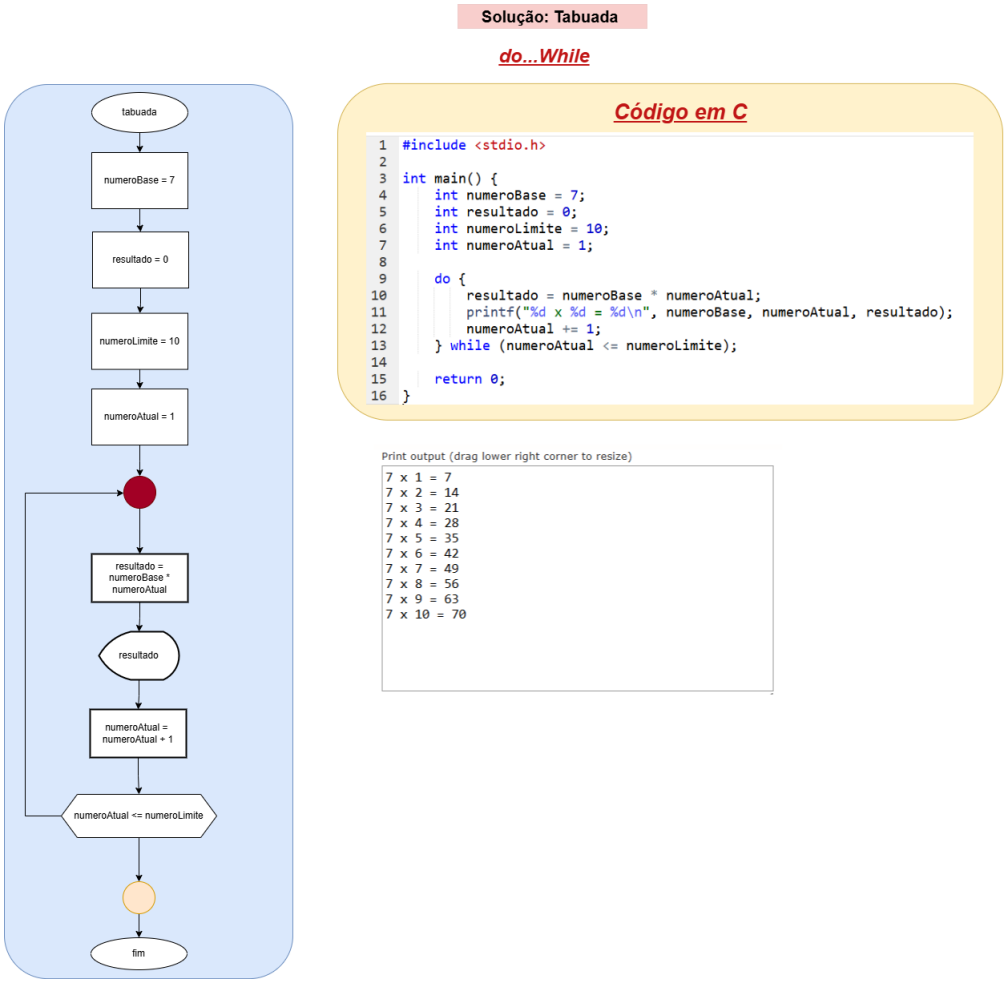


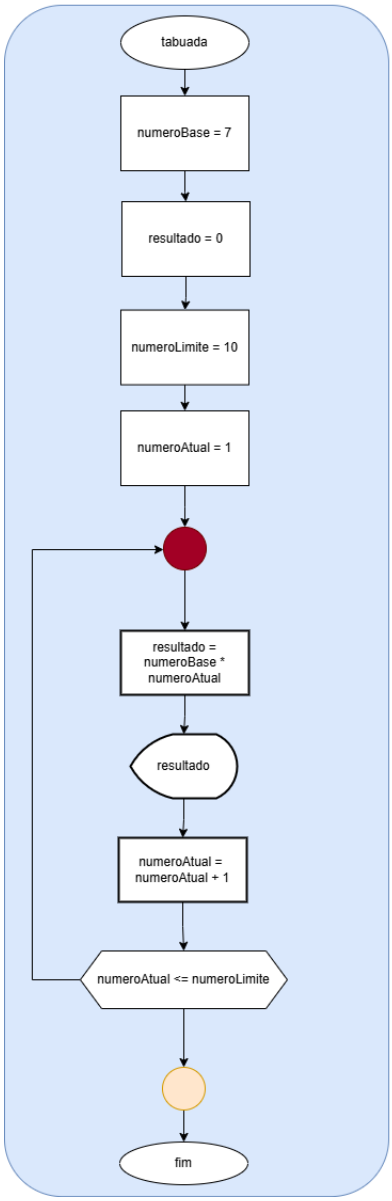
Figura 11: Diagrama, Código C e Resultado - Laço `WHILE`

3.3 Exemplo com a Estrutura `faca...enquanto` (do...while)

Aqui, o programa executa o cálculo da tabuada primeiro e testa a condição ao final. É garantido que pelo menos a operação 7×1 será realizada antes de qualquer verificação.

Solução: Tabuada

do...While



Código em C

```c
#include <stdio.h>

int main() {
    int numeroBase = 7;
    int resultado = 0;
    int numeroLimite = 10;
    int numeroAtual = 1;

    do {
        resultado = numeroBase * numeroAtual;
        printf("%d x %d = %d\n", numeroBase, numeroAtual, resultado);
        numeroAtual += 1;
    } while (numeroAtual <= numeroLimite);

    return 0;
}
```

Print output (drag lower right corner to resize)

```
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
```

Figura 12: Diagrama, Código C e Resultado - Laço `DO...WHILE`

Análise Comparativa

Observe que, embora as sintaxes mudem, o resultado final (output) é idêntico nos três casos. A escolha da estrutura depende da legibilidade do código e da necessidade de testar a condição no início ou no fim do processamento.